


# Atividades Práticas

Trilha Dominando Algoritmos:
Estrutura de Dados Vetor

# Análise de Desempenho de uma Turma

Uma escola deseja analisar o desempenho final dos alunos de uma turma com base nas notas obtidas ao final do período letivo.

Elabore um algoritmo que:

- Leia a quantidade de alunos da turma;
- Receba a nota final de cada aluno (valores de 0 a 10), armazenando-as em um vetor;

Ao final, o programa deverá exibir:

- Todas as notas informadas;
- A **média** da turma;
- A **maior** e a menor nota;
- A **quantidade** de alunos aprovados (nota ≥ 6) e reprovados (nota < 6);
- Uma **mensagem** indicando:
  - *"Todos os alunos foram aprovados"*, ou
  - *"Nem todos os alunos foram aprovados"*.

**Cenários de Testes**

| Entrada | Saída |
|---|---|
| 3<br>6<br>7<br>8 | >> Resultado da Turma <<<br>Notas informadas: 6, 7, 8<br><br>📈 Média da turma: 7<br>🏆 Maior nota: 8<br>🔻 Menor nota: 6<br>✅ Aprovados: 3<br>❌ Reprovados: 0<br><br>Situação geral: Todos os alunos foram aprovados |

# Análise Mensal de Faturamento

Uma empresa deseja analisar o comportamento do seu faturamento ao longo dos meses para identificar tendências de crescimento ou queda.

Desenvolva um programa que leia a quantidade de meses analisados e, em seguida, receba os valores de faturamento mensal, armazenando-os em um vetor.

Ao final, o programa deverá analisar a sequência dos valores e informar se o faturamento da empresa está:

- **Crescimento**: cada mês faturou igual ou mais que o anterior;
- **Queda:** cada mês faturou igual ou menos que o anterior;
- **Constante:** todos os meses possuem o mesmo faturamento;
- **Sem padrão**: não segue nenhum dos padrões acima.

**Cenários de Testes**

| Entrada | Saída |
|---|---|
| 5<br>1000<br>1500<br>2000<br>2000<br>2100 | >> Análise de Faturamento <<<br><br>💰 Faturamento informado: 1000, 1500, 2000, 2000, 2100<br>📉 Situação: Crescimento |
| 4<br>3000<br>3000<br>2500<br>2000 | >> Análise de Faturamento <<<br><br>💰 Faturamento informado: 3000, 3000, 2500, 2000<br>📉 Situação: Queda |
| 2<br>3000<br>3000 | >> Análise de Faturamento <<<br><br>💰 Faturamento informado: 3000, 3000<br>📉 Situação: Constante |

# Doações

Um **projeto de caridade** deseja acompanhar as doações recebidas ao longo de uma campanha, com o objetivo de verificar seu desempenho em relação à meta estabelecida.

**Desenvolva um algoritmo que:**

- Leia o nome do projeto de caridade;
- Leia a quantidade de doações recebidas;
- Leia o valor da meta financeira a ser atingida;
- Solicite o valor de cada doação, armazenando-os em um vetor;

**Ao final, o programa deverá:**

- Exibir todos os valores das doações realizadas;
- Calcular e mostrar o total arrecadado;
- Identificar e exibir o maior valor de doação recebido;
- Informar se a meta foi atingida ou não, com base no total arrecadado.

**Cenários de Testes**

| Entrada | | Saída |
|---|---|---|
| Asilo Lar Jésus | *// projeto* | **>> Resumo das Doações <<** |
| 3 | *// qtd de doações* | |
| 800 | *// meta* | **Projeto: Asilo Lar Jésus** |
| 100 | *// doações..* | **Doações recebidas: 100, 200, 150** |
| 200 | | |
| 150 | | **Total arrecadado: 450** |
| | | **Maior doação: 200** |
| | | **Situação: Meta não atingida** |

# FarmaZoom

O sistema **FarmaZoom** foi criado para ajudar consumidores a economizar na compra de medicamentos, comparando os preços em duas farmácias diferentes.

Desenvolva um programa que leia a quantidade de medicamentos e, em seguida, receba os preços desses medicamentos na **Farmácia A** e na **Farmácia B**, armazenando-os em dois vetores.

Ao final, o programa deverá:

- Exibir os preços dos medicamentos em cada farmácia;
- Comparar os preços medicamento por medicamento e informar:
  - Quantos estão mais baratos na Farmácia A;
  - Quantos estão mais baratos na Farmácia B;
  - Quantos possuem o mesmo preço;
- Calcular o valor total da compra em cada farmácia;
- Informar em qual farmácia a compra total seria mais vantajosa, ou se há empate.

**Cenários de Testes**

| Entrada | Saída |
|---|---|
| 4 *// qtd*<br>20 *// preços Farmácia A*<br>35<br>50<br>40<br>25 *// preços Farmácia B*<br>30<br>50<br>38 | 💊 **FARMAZOOM**<br>**Farmácia A: 20, 35, 50, 40**<br>**Farmácia B: 25, 30, 50, 38**<br><br>💸 **Produtos mais Baratos**<br>**Na Farmácia A: 1**<br>**Na Farmácia B: 2**<br>**Mesmo preço: 1**<br><br>💰 **Total**<br>**Farmácia A: R$ 145**<br>**Farmácia B: R$ 143**<br><br>🏆 **Melhor opção**<br>**Farmácia B** |

# Jogo da Forca

Implemente um jogo da forca simplificado, no qual uma palavra secreta é representada por um vetor de caracteres.

**O algoritmo deve:**

- Ler a palavra secreta e armazená-la em um vetor (*pule linhas para que a palavra não fique visível no console*);
- Criar um vetor auxiliar, do mesmo tamanho, inicialmente preenchido com _ (underline), representando as letras ainda não descobertas;
- Permitir que o jogador informe letras uma a uma;
- A cada tentativa:
  - Verificar se a letra informada existe na palavra;
  - Caso exista, atualizar o vetor auxiliar revelando todas as ocorrências da letra;
  - Caso não exista, contabilizar um erro;
- Exibir, a cada rodada:
  - O estado atual da palavra (com letras descobertas e _ para as ocultas);
  - A quantidade de erros cometidos;
- O jogo deve terminar quando:
  - O jogador descobrir todas as letras da palavra (vitória), ou
  - Atingir um número máximo de 7 erros (derrota).

**Cenários de Testes**

| Entrada | Saída |
|---|---|
| ASA<br>A<br>S | Forca: A _ A<br>Erros: 0<br><br>Forca: A S A<br>Erros: 0<br><br>Parabéns! Você descobriu a palavra. |
| LEI<br>L<br>E<br>R<br>U<br>N<br>O<br>A<br>D<br>A | Estado: L _ _<br>Erros: 0<br><br>Estado: L E _<br>Erros: 0<br><br>Estado: L E _<br>Erros: 1<br><br>*// erros 2, 3, 4, 5 e 6*<br><br>Estado: L E _<br>Erros: 7<br><br>Você perdeu! A palavra era: LEI |

# Projeto Fitness

Uma pessoa está acompanhando seu peso diariamente e registrando os valores ao longo de vários dias. Com base nesses dados, deseja-se analisar a variação do peso e visualizar essa evolução de forma simples no terminal.

**Elabore um algoritmo que:**

- Leia a quantidade de dias monitorados;
- Receba os pesos correspondentes a cada dia e armazene em um vetor;
- Exiba todos os pesos informados;
- Identifique o maior peso e o dia em que ocorreu;
- Identifique o menor peso e o dia em que ocorreu;
- Mostre um gráfico de barras no terminal, representando o peso de cada dia.

**No gráfico:**

- Cada barra deve ser formada pelo caractere ▉;
- Cada ▉ representa 5 kg;
- O valor exato do peso deve ser exibido ao lado da barra.

**Cenários de Testes**

| Entrada | Saída |
|---|---|
| 4 | Pesos informados: 70, 72, 68, 75 |
| 70 | Maior peso: 75 kg (Dia 4) |
| 72 | Menor peso: 68 kg (Dia 3) |
| 68 | |
| 75 | Gráfico de evolução: |
| | Dia 1 \| ▉▉▉▉▉▉▉▉▉▉▉▉▉▉ 70 |
| | Dia 2 \| ▉▉▉▉▉▉▉▉▉▉▉▉▉▉ 72 |
| | Dia 3 \| ▉▉▉▉▉▉▉▉▉▉▉▉▉ 68 |
| | Dia 4 \| ▉▉▉▉▉▉▉▉▉▉▉▉▉▉▉ 75 |

# Histórico de Transações em Conta Corrente

Um sistema bancário deseja registrar o histórico de transações realizadas em uma conta corrente ao longo de um período. Desenvolva um algoritmo que permita ao usuário inserir valores de transações, **obedecendo às seguintes regras:**

- O programa deve permitir o registro de até 1000 operações;
- Cada valor digitado representa uma transação:
  - Valores positivos representam depósitos;
  - Valores negativos representam saques;
- O valor 0 possui um comportamento especial:
  - Ao digitar 0, a última transação válida deve ser cancelada;
  - Se forem digitados dois ou mais 0 consecutivos, o programa deve continuar cancelando transações anteriores;
  - Caso a última operação já seja um cancelamento, o programa deve retroceder ainda mais, cancelando a última transação válida anterior;
- O valor -1 indica o fim da entrada de dados.

**Ao final, o programa deverá:**

- Exibir o saldo final da conta corrente, considerando todas as transações válidas após os cancelamentos.

**Cenários de Testes**

| Entrada | Saída |
|---|---|
| 100<br>50<br>-20<br>0<br>-1 | **Saldo final: 200** |
| 200<br>-50<br>30<br>0<br>0<br>-1 | **Saldo final: 200** |
| 100<br>50<br>0<br>0<br>0<br>-1 | **Saldo final: 0** |

# Controle de Estoque

Uma loja deseja controlar seu estoque de produtos por meio de um sistema simples, que permita registrar produtos e atualizar suas quantidades ao longo do tempo. Desenvolva um algoritmo que utilize um menu de opções para gerenciar o estoque.

**O programa deve:**

- Permitir o cadastro de até 1000 produtos;
- Armazenar os nomes dos produtos e suas respectivas quantidades em estoque (*use dois vetores para isso*);
- Exibir um menu continuamente, até que o usuário escolha encerrar o programa.

| Menu de Opções |
| --- |
| 1. Adicionar produto |
| 2. Realizar entrada no estoque |
| 3. Realizar saída no estoque |
| 4. Listar estoque |
| 0. Sair |

**Regras do Algoritmo:**

- O programa deve impedir o cadastro de novos produtos caso o limite de 1000 produtos seja atingido;
- Para operações de entrada e saída, o usuário deve informar o número do produto;
- Não permitir retirada de quantidade maior do que a disponível em estoque;
- Caso não existam produtos cadastrados, o programa deve informar ao usuário;
- O menu deve ser exibido novamente após cada operação, até a escolha da opção de saída.

**Cenários de Testes**

| Entrada | Saída |
| --- | --- |
| 1 *(opcao)*<br>Arroz<br>10 | Produto adicionado com sucesso!<br>Produto adicionado com sucesso!<br><br>Entrada realizada com sucesso! |
| 1 *(opcao)*<br>Feijão<br>5 | Saída realizada com sucesso! |
| 2 *(opcao)*<br>1 *(produto)*<br>5 | Estoque atual:<br>Produto 1: Arroz - 15 unidades<br>Produto 2: Feijão - 2 unidades |
| 3 *(opcao)*<br>2 *(produto)*<br>3 | Encerrando o programa... |
| 4 *(opcao)*<br>0 *(sair)* | |

# Controle de Tarefas

Uma pessoa deseja organizar melhor suas atividades diárias e precisa de um programa que permita gerenciar suas tarefas ao longo do dia de forma simples e interativa. Desenvolva um algoritmo que utilize um menu de opções para permitir o gerenciamento dessas tarefas.

**O programa deve:**

- Criar uma estrutura (vetor) para armazenar as tarefas do dia;
- Exibir repetidamente um menu com opções, permitindo ao usuário escolher a ação desejada;
- Executar a operação escolhida até que o usuário decida encerrar o programa.

| Menu de Opções |
|---|
| 1. Adicionar tarefa |
| 2. Listar tarefas |
| 3. Marcar concluída |
| 4. Exibir pendentes |
| 0. Sair |

**Regras do Algoritmo:**

- O menu deve ser exibido continuamente até que o usuário escolha a opção de saída;
- As tarefas devem ser armazenadas em um vetor;
- Caso não existam tarefas cadastradas, o programa deve informar ao usuário;
- O programa deve validar as opções escolhidas no menu, exibindo mensagem de erro para opções inválidas.

**Cenários de Testes**

| Entrada | Saída |
|---|---|
| 1 *(opcao)*<br>Estudar Lógica | Tarefa adicionada com sucesso!<br>Tarefa adicionada com sucesso!<br>Tarefa adicionada com sucesso! |
| 1 *(opcao)*<br>Fazer Exercícios | Lista de tarefas:<br>1 - [ ] Estudar Lógica<br>2 - [ ] Fazer exercícios<br>3 - [ ] Ler um livro |
| 1 *(opcao)*<br>Ler um Livro | |
| 2 *(opcao)* | Tarefa 2 marcada como concluída. |
| 3 *(opcao)*<br>2 | Lista de tarefas:<br>1 - [ ] Estudar Lógica<br>2 - [X] Fazer exercícios<br>3 - [ ] Ler um livro |
| 2 *(opcao)* | |
| 4 *(opcao)*<br>0 *(sair)* | Tarefas pendentes:<br>1 - [ ] Estudar Lógica<br>3 - [ ] Ler um livro |
| | Encerrando o programa... |

# Troca de figurinhas

Dois jogadores possuem coleções de figurinhas e desejam realizar trocas entre si para completar seus álbuns.

**Desenvolva um algoritmo que:**

- Leia a quantidade de figurinhas do primeiro jogador;
- Leia os números das figurinhas do primeiro jogador, armazenando-os em um vetor;
- Leia a quantidade de figurinhas do segundo jogador;
- Leia os números das figurinhas do segundo jogador, armazenando-os em outro vetor;

**Ao final, o programa deverá:**

- Identificar e exibir as figurinhas que ambos possuem (repetidas entre os dois);
- Identificar e exibir as figurinhas que apenas o primeiro jogador possui;
- Identificar e exibir as figurinhas que apenas o segundo jogador possui;
- Calcular e exibir a quantidade de trocas possíveis, considerando que:
    - Cada jogador só pode oferecer figurinhas que o outro não possui;
    - Uma troca ocorre quando cada jogador tem pelo menos uma figurinha que o outro não tem.

**Cenários de Testes**

| Entrada | Saída |
|---|---|
| 5 *(jogador1)*<br>1<br>2<br>3<br>4<br>5<br><br>4 *(jogador2)*<br>3<br>4<br>6<br>7 | **Figurinhas em comum: 3, 4**<br><br>**Apenas jogador A: 1, 2, 5**<br>**Apenas jogador B: 6, 7**<br><br>**Quantidade de trocas possíveis: 2** |